

Project Proposal: Maze Solver Using State Space Tree and Shortest Path Algorithm

1. Project Title: Maze Solver Using State Space Tree and Shortest Path Algorithm

2. Team Members: Syed Sualeh Abbas, Syed Arham Hussain, Abdullah Naeem

3. Project Overview: This project aims to develop a maze solver that efficiently finds the shortest path in a given maze represented as a matrix. The maze will first be converted into a state space tree, and then an optimal path will be determined using an appropriate shortest path algorithm.

4. Problem Statement: Maze-solving is a classical problem in computer science and artificial intelligence. Finding the shortest path in a maze is essential for applications in robotics, pathfinding, and artificial intelligence. The goal of this project is to implement an efficient algorithm that converts a maze into a structured state space tree and then determines the shortest path from the start position to the goal.

5. Objectives:

- Represent the maze as a matrix.
- Convert the matrix into a state space tree. A matrix can be converted into a **state space tree** by treating each cell as a node and defining valid movements as edges.
- Implement an algorithm to find the shortest path efficiently.
- Visualize the path from the start node to the goal node.
- Analyze the time and space complexity of the implemented algorithm.

6. Methodology:

1. Maze Representation:

- The maze will be represented as a 2D matrix where:
 - 0 represents open paths.
 - 1 represents obstacles/walls.
 - S represents the start position.
 - G represents the goal position.

2. State Space Tree Generation:

- Each possible movement (0) (left, right, up, down) will be considered as a node in the state space tree.
- A tree data structure will be used to store the paths explored.

3. Shortest Path Algorithm:

- The algorithm will backtrack if a dead-end is encountered and will continue exploring the shortest path.

4. Implementation:

- The program will take the maze matrix as input.
- It will generate the state space tree and apply the shortest path algorithm.
- The final solution will be presented as a marked matrix showing the optimal path.

7. Testing:

- Various test cases with different maze structures will be used.

8. Expected Outcome:

- A functional maze solver capable of finding the shortest path.
- A visual representation of the solved maze.
- A performance comparison of different pathfinding algorithms.

9. Tools & Technologies:

- Programming Language: Python
- Data Structures: State Space Trees, 2D matrix

10. Project Timeline:

- **Week 1:** Research and algorithm selection.
- **Week 2:** Implementation of maze representation and state space tree.
- **Week 3:** Implementation of the shortest path algorithm.
- **Week 4:** Testing, debugging.

11. Conclusion: This project will provide an in-depth understanding of data structures, and algorithm design.
